

Volga Can Karakus - Portfolio

Description:

Most of my professional work is proprietary, either shipped commercial products or owned by former employers, so I can't share source code or internal materials. Instead, this portfolio presents case studies of selected projects. Each one covers the problem, the system architecture, the engineering challenges, and the outcome, along with my specific role and the technologies involved.

Overview:

A native Android command-center application that let field operators monitor and manually drive autonomous robots in real time, visualize robot position and mission geometry on a live map, execute remote commands, and deliver robot data and logs to the cloud. It was deployed to operators across the USA, Canada, Malaysia, Brazil, India, and France.

My Contribution

I was responsible for the end-to-end development of both the mobile application and the robot-side software, including the communication layer connecting the two systems.

- **Architecture and design:** Designed the system architecture, splitting the robot backend into a low-latency C++ control service and a secure Node.js web service. Produced technical diagrams and proof-of-concept demos, presented them to management and stakeholders, and refined the design based on their feedback before implementation.
- **Mobile application:** Built the native Android app (Java, AndroidX) with live map visualization of robot position, paths, waypoints, and work areas using the Google Maps SDK. Kept the UI responsive during manual driving by moving parsing, logging, and network I/O off the main thread on low-power tablets.
- **Real-time communication layer:** Designed and implemented the event and listener systems between the tablet and robot, using raw WebSockets to the C++ service (uWebSockets, binary payloads) for telemetry and control, and Socket.IO to the Node.js service for upload progress and verification status.
- **Robot-side services and APIs:** Developed Node.js/Express API endpoints for data storage, validation, and retrieval, with session-based authentication backed by MongoDB and bcrypt password hashing.
- **Secure, verified cloud uploads:** Designed authenticated multipart upload workflows to AWS S3 using short-lived STS and Cognito credentials, so no permanent keys lived on robots or tablets. Ensured data integrity over unreliable connections with SHA-256 hashing before upload, the hash is given as a parameter to the independent re-verification by an S3-triggered Lambda after upload, and handling of interrupted or incomplete transfers.
- **Deployment and version management:** Containerized robot-side services with Docker and implemented mechanisms to verify and synchronize the Docker image versions running on each robot against the registry.
- **Remote operations and safe shutdown:** Implemented remote command execution over SSH and controlled shutdown procedures that safely stopped robot-side devices and services before power was disconnected, preventing data corruption and hardware faults.

Technical Architecture:

- **On robot side (Server 1)** : Performance-critical and reliability-oriented architecture designed for low-latency data transfer and processing. Only the tablet client connects to this server.
 - **Implementation:** C++ with uWebSockets, using binary payloads and UDP over Ethernet for efficient communication.
 - **Hardware interfaces:** Serial/USB-UART communication for LiDAR/camera systems.
 - **Deployment:** Fully containerized using Docker.
 - **Evolution:** Later migrated toward a ROS 2-based architecture.

- **On robot side (Server 2):** Secure, web-facing service for operator authentication, large-file uploads from the robot, and verified cloud storage. Tablets or any other device can connect to this server.
 - **Implementation:** Node.js with Express and server-rendered EJS views, using Socket.IO for real-time upload progress and verification status.
 - **Authentication:** Session-cookie login backed by MongoDB (Mongoose), with bcryptjs password hashing and Mongoose schema validation on user input.
 - **Cloud storage:** Multipart uploads to AWS S3 via @aws-sdk/client-s3 and @aws-sdk/lib-storage, with bucket CORS configured for browser access.
 - **Access control:** AWS STS-issued temporary credentials scoped for uploads and Lambda execution, avoiding long-lived keys on the device.
 - **Data integrity:** Full-file SHA-256 computed before upload and stored as S3 object metadata, then re-verified by an S3-triggered Lambda that streams the object and recomputes the hash using Node's crypto library.
 - **Data handling:** JSON payload parsing and processing of robot data files.
 - **Deployment:** Fully containerized using Docker.

- **On Tablet Side (Client)** : Operator-facing mobile application for real-time robot monitoring and control, map-based visualization of robot position and mission geometry, remote command execution, and automated log delivery to cloud storage.
 - **Implementation:** Native Android app written in Java 8 on the Android SDK and AndroidX, built with Gradle.
 - **Real-time communication:** Socket.IO connection (socket.io-client-java) to the Node.js server for live status and upload events. A raw WebSocket connection to the C++ robot service runs through OkHttp's WebSocket and WebSocketListener.
 - **Remote execution:** Shell commands run on the robot over SSH using JSch exec channels.
 - **Mapping:** Google Maps SDK for Android renders robot and waypoint markers, travelled and planned paths as polylines, and work areas and boundaries as polygons. android-maps-utils provides geometry calculations and overlay handling.
 - **Location:** Fused Location Provider supplies device positioning, and a location-type foreground service keeps tracking continuous. Location and storage

access are handled through manifest declarations and runtime permission requests.

- **Data handling:** WebSocket messages and robot data payloads are parsed and serialized with org.json and Gson.
- **Concurrency:** Handler/Looper posting results to the UI thread. WorkManager handles deferred, retryable background jobs.
- **Logging:** A custom file logger writes structured logs to app-specific storage, compliant with Android scoped storage.
- **Cloud storage:** Log files are uploaded to AWS S3 through the AWS SDK for Android. Uploads authenticate with Amazon Cognito-issued temporary credentials, so no long-lived keys are embedded in the app.

Scale & Constraints:

- **Global operator base:** Robots and operators deployed across multiple countries (USA, Canada, Malaysia, Brazil, India, France). Network quality, latency to cloud regions, and connectivity reliability varied widely by site.
- **Asymmetric hardware:** The robot's onboard compute (CPU/board) far outpaced the low-power Android tablets used by operators, yet the tablet had to consume the robot's full real-time data stream.
- **Real-time safety requirement:** Operators drove the robot manually from the tablet. Any UI stall translated directly into delayed control input, so a frozen screen was a safety and reliability risk.
- **Concurrent data load:** The tablet simultaneously handled high frequency messages of telemetry (position, LiDAR/camera-derived status), map rendering, GPS tracking, upload progress events, and SSH command results.
- **Large data volumes:** Robot data files of each being around 1GB to 30GB had to reach cloud storage with verified integrity, often over unreliable field connections.
- **Security in the field:** Devices operated outside controlled environments, so long-lived credentials on robots or tablets were not acceptable.

Engineering Challenges:

- **Producer/consumer mismatch:** The robot could generate data faster than the tablet could parse and render it. Processing that load naively on the main thread would freeze the UI mid-drive. Each message was sent/received with unique UUIDs and the display values would only be changed if actual changes were involved in the data received by the tablet.
- **Keeping the control path responsive:** Manual driving commands and the live robot position had to stay low-latency even while heavy work (map overlays, JSON parsing, file logging, uploads) competed for the same limited CPU.
- **Reliable uploads across regions:** Large multipart uploads had to survive interruptions and variable bandwidth across sites and still guarantee the stored file matched the original content.

- **Credential management at scale:** Many devices across many regions needed upload and Lambda access without embedding or distributing permanent keys.
- **Separating critical and non-critical traffic:** Low-latency control data and heavyweight, secure web traffic (auth, uploads) have opposite requirements and could not share one service without one degrading the other.

Approach & Implementation:

- **Split the robot backend by responsibility.** A performance-critical C++ service handles real-time control and telemetry, exclusive to the tablet. A separate Node.js service handles authentication, uploads, and cloud verification. Heavy or failure-prone work therefore never blocks the control path.
- **Reduced payload cost at the source.** Binary payloads and UDP over Ethernet on the robot side minimized bandwidth and parsing overhead before data ever reached the tablet.
- **Kept the tablet's main thread for UI only.** Parsing, logging, and network I/O run off the main thread, and only final results are posted back to the UI. Deferred work like log uploads goes to WorkManager so it retries without operator involvement.
- **Built in end-to-end integrity verification.** A SHA-256 hash is computed before upload, stored with the object, and independently recomputed by a Lambda after arrival, so corrupted or partial uploads are detected automatically.
- **Used short-lived, scoped credentials everywhere.** STS on the robot and Cognito on the tablet mean no permanent keys exist on field devices.
- **Containerized for repeatable deployment.** Docker on both robot services gave consistent deployments across fleets in every region and eased the later migration toward ROS 2.

Outcome:

- **Performance:** Eliminated UI freezes during manual driving; reduced frame drops / input latency sustaining high frequency msgs/sec on low-end tablets.
- **Reliability:** Near 100% upload success rate across all regions, with automatic integrity verification catching corrupted transfers before they caused data issues; failed uploads recovered via automatic retry.
- **Security:** Zero long-lived credentials on robots or tablets, which reduced the impact of a lost or compromised device.
- **Cost:** Reduced bandwidth/storage/compute costs through binary payloads.
- **Operational efficiency:** Automated log delivery removed manual log collection, cutting field support and error.
- **Maintainability:** Containerized, decoupled services enabled the incremental migration to ROS 2 without rewriting the tablet client.

Overview:

A cloud-based analytics platform for agricultural field data. Users requested analysis of their fields, ran compute-heavy pipelines on demand or in scheduled batches, and explored results through an interactive field/plot map. It started as a user-triggered, per-request compute system and was later redesigned into a queued, spot-instance batch system to cut infrastructure costs. Lightweight analyses ran serverless on AWS Lambda. Deployed to the USA, Canada, Malaysia, Brazil, India, and France.

My Contribution:

I designed the system architecture end to end, including the web application, the data ingestion and serverless processing layer, the job queueing system, and the automated provisioning of analysis instances.

- **Web application:** Built the Meteor application with a React front end, using DDP (Distributed Data Protocol) publications and subscriptions so users saw request status and results update in real time without page refreshes.
- **Interactive visualization:** Built the field/plot grid with D3.js and SVG. It represented each user's field and its individual plots and let users select specific field/plot and data-type combinations to view results and tables.
- **Data delivery:** Implemented server-side ZIP generation with Node streams (archiver). Files were streamed straight from S3 into the archive and out to the user, so large downloads were never fully loaded into server memory.
- **Serverless ingestion and processing:** Built S3-triggered AWS Lambda functions that archived newly uploaded raw data as compressed ZIP copies, extracted metadata from incoming data into MongoDB, and ran lightweight analytics such as LiDAR-based plant height estimation without provisioning any EC2 instance.
- **Job orchestration:** Designed the request and job queue. MongoDB tracked requests, job state, and result metadata, and SQS dispatched jobs to workers. Worker EC2 instances were provisioned programmatically, sized to each request's needs, and terminated automatically when the job completed.
- **Analysis pipeline infrastructure:** Built custom AMIs with the analysis algorithms baked in. EC2 user data / cloud-init scripts started the pipeline on boot and passed in job parameters from the customer's queue.
- **Cost redesign:** Led the move from on-demand, per-request instances to queued batch processing on EC2 Spot Instances via EC2 Fleet. Batched jobs in bulk to raise instance utilization, and made jobs safe to retry or requeue when a spot instance was interrupted.
- **Deployment and infrastructure:** Deployed the app with Meteor Up (mup) and Docker onto EC2, behind an Elastic Load Balancer configured for WebSocket support so DDP connections stayed stable. Auto Scaling, Route 53 for DNS, and ACM certificates for TLS completed the setup.

- **Security and access:** Defined IAM roles and policies for app servers and worker instances. Implemented API-based login and STS temporary credentials for special-case access, so no long-lived keys were needed.

Technical Architecture:

- **Web Application (Frontend):** User-facing interface for creating analysis requests, tracking their status, and exploring results.
 - **Implementation:** React on Meteor, with HTML5, CSS3, and JavaScript (ES6+).
 - **Visualization:** D3.js rendering an SVG grid of the user's field and plots, with interactive selection of field/plot and data-type combinations, plus D3-generated result tables.
 - **Real-time updates:** Meteor DDP publications and subscriptions push request status and new results to the client as they arrive.
- **Web Application (Backend):** Request handling, data access, and file delivery.
 - **Implementation:** Meteor on Node.js, with Meteor Methods for client-server calls.
 - **Database:** MongoDB storing users, fields, plots, data types and structures, request/job state, and queryable result summaries.
 - **Cloud access:** AWS SDK for JavaScript for S3 and EC2.
 - **File delivery:** archiver with Node streams, zipping S3 objects on the fly while streaming them to the user.
 - **Authentication:** API calls for login and for issuing STS temporary credentials in special use cases.
- **Serverless Processing (AWS Lambda):** Event-driven ingestion and lightweight analytics.
 - **Archiving:** S3 upload events trigger Lambda functions that store a compressed ZIP copy of the raw data alongside the original.
 - **Metadata extraction:** Lambda parses incoming data and writes metadata entries to MongoDB, so new data is immediately queryable and shows up in the field/plot grid.
 - **Lightweight analytics:** Quick, CPU-only jobs such as plant height estimation from LiDAR data run directly in Lambda, with results written to S3 and MongoDB.
- **Compute and Analysis Pipeline:** Automated processing of large field datasets, split into two tiers by workload size.
- **Routing:** Lightweight, CPU-only analyses run on Lambda. Heavy or GPU-dependent pipelines are queued for EC2 workers.
- **Job queue:** SQS-backed job dispatch, with MongoDB as the source of truth for request and job status.
- **Workers:** EC2 instances launched from custom AMIs that contain the analysis algorithms. Instance type is chosen per request.
- **Bootstrapping:** EC2 user data / cloud-init scripts start the pipeline and pass in job parameters.

- **Data flow:** Input data is streamed from S3 into the processing instances. Raw and bulk outputs are written to S3, and summaries go to MongoDB for fast querying and display.
- **Lifecycle:** Instances terminate automatically when their job completes.
- **Evolution:** Migrated from per-request on-demand instances to batched processing on Spot Instances via EC2 Fleet, with retry and requeue on spot interruption.

Infrastructure and Deployment:

- **Deployment:** Meteor Up (mup) with Docker containers on EC2 app servers.
- **Networking:** Elastic Load Balancer with WebSocket support for DDP, Route 53 for DNS, and AWS Certificate Manager for SSL/TLS termination.
- **Scaling:** EC2 Auto Scaling for app servers, driven by CloudWatch alarms.
- **Access control:** IAM roles and policies scoped separately for app servers and worker instances.
- **Monitoring:** CloudWatch for logs, metrics, and scaling alarms.

Scale & Constraints:

- **Large datasets:** Field datasets of 1GB to 30GB requests had to be processed and delivered without exhausting app server memory.
- **Variable workloads:** Requests varied widely in size and compute needs, so a single fixed instance type would either waste money or underperform.
- **Long-running jobs:** Analysis jobs ran for [minutes/hours] and could not run on the web servers without degrading the user experience.
- **Cost pressure:** Launching on-demand instances per user request became expensive as usage grew.
- **Real-time UX:** Users expected live status updates and responsive interaction with the field/plot grid, which depended on persistent WebSocket connections through a load balancer.

Engineering Challenges:

- **Separating web traffic from heavy compute:** Analysis pipelines are CPU- and I/O-heavy and long-running. Running them anywhere near the web tier would have made the app slow or unresponsive.
- **Right-sizing compute per request:** Each request needed a different amount of compute, and provisioning had to be automatic, with no manual instance management.
- **Matching compute to the job:** Not every analysis justified a full EC2 instance. Spinning one up for a quick LiDAR-only calculation added boot time and cost far beyond the work itself.
- **Making new data usable immediately:** Raw uploads had to be archived, indexed, and visible in the application without manual steps.
- **Large downloads without memory blowup:** Users needed to download many large result files at once, and buffering them on the server was not viable.
- **Keeping DDP stable behind a load balancer:** Meteor's real-time layer depends on long-lived WebSocket connections, which load balancers and scaling events can easily break.

- **Controlling compute cost:** Per-request on-demand instances left capacity idle and drove costs up as usage grew.
- **Surviving spot interruptions:** Moving to Spot Instances meant any job could be terminated mid-run, and no request could be lost or left half-finished.

Approach & Implementation:

- Decoupled the web tier from compute. The Meteor app only records requests and enqueues jobs. Dedicated worker instances launched from custom AMIs do all the heavy analysis, so it never touches the web servers.
- Generated compute by workload. Short, CPU-only jobs like plant height from LiDAR ran on Lambda with no instance startup cost. Heavy and GPU-dependent pipelines went through the SQS queue to EC2 workers.
- Provisioned compute per job. Jobs carry their parameters, the system picks an appropriate instance type, cloud-init starts the pipeline on boot, and the instance terminates itself when finished, so there is no idle capacity between jobs.
- Automated ingestion with events. S3-triggered Lambdas archived raw uploads as ZIPs and wrote metadata to MongoDB as data arrived, so it was immediately queryable and ready for analysis.
- Streamed data end to end. Input data streams from S3 into workers. Downloads stream from S3 through archiver directly to the user, so memory use stays flat regardless of archive size.
- Stored results for two access patterns. Bulk outputs live in S3 for cheap storage and download. Summaries live in MongoDB so the D3 grid can query specific field/plot combinations quickly.
- Configured the load balancer for WebSockets. The ELB was set up to support DDP's persistent connections, and Auto Scaling kept the app tier sized to demand.
- Redesign around batching and Spot. Moved from user-triggered instances to queued batch processing on EC2 Fleet Spot Instances. Jobs were grouped to maximize utilization and made safe to retry, so interrupted jobs were requeued automatically.
- Used scoped, short-lived access. IAM roles separated app and worker permissions, and STS temporary credentials covered special access cases without distributing permanent keys.

Outcome:

- **Cost:** Reduced compute costs by moving from on-demand per-request instances to batched Spot Instance processing due to user inconsistency in request batching.
- **Efficiency:** Lightweight analyses returned results in seconds to couple minutes on Lambda instead of waiting for EC2 provisioning, and incurred no idle instance cost.
- **Automation:** Uploaded data was archived and indexed automatically, with no manual processing between upload and availability.
- **Scalability:** The platform handled requests with no manual instance management.
- **Reliability:** Spot interruptions were handled automatically through requeue, with no lost jobs.

- **Performance:** Streaming ZIP generation kept app server memory flat on downloads of any size, and the web tier stayed responsive while analysis ran on separate workers.
- **User experience:** Real-time status updates and an interactive field/plot grid let users go straight to the results they needed.
- **Maintainability:** Baking the algorithms into AMIs made pipeline updates a matter of publishing a new image, with no changes to the web application.

Overview

My main objective for this project was to challenge myself to learn a new language and new technologies such as the Unity game engine while improving my skills of prompting AI tools such as Claude Code. I am the sole owner of this project and the game is published on the IOS app store and waiting for the final approval on Android Play Store.

Shroomverse is a cross-platform idle/strategy game fully designed by me, built in Unity 6 (URP) in C#.

The player grows a mushroom colony on a procedurally generated planet, expands across a galaxy of independent planet economies, builds production chains from raw ore to capital ships, and fights asynchronous battles against other AI generated players covered by unit tests to make the fights fair depending on the level of the achievements.

The game ships in 29 languages, including four right-to-left languages, and carries in-app purchases, rewarded ads, push notifications and crash reporting on both stores.

Applying my knowledge and learning new skills made this a very informative and entertaining experience.

Project Diagrams :

- [Progression Diagram](#)
- [Buildings Diagram](#)
- [Research Tree Diagram](#)
- [Materials Diagram](#)
- [University Research Tree Diagram](#)
- [Units Combat Diagram](#)

My Contribution

I designed and built the whole game as a solo developer: gameplay systems, simulation, UI, persistence, multiplayer, monetization, localisation and the native iOS/Android integration.

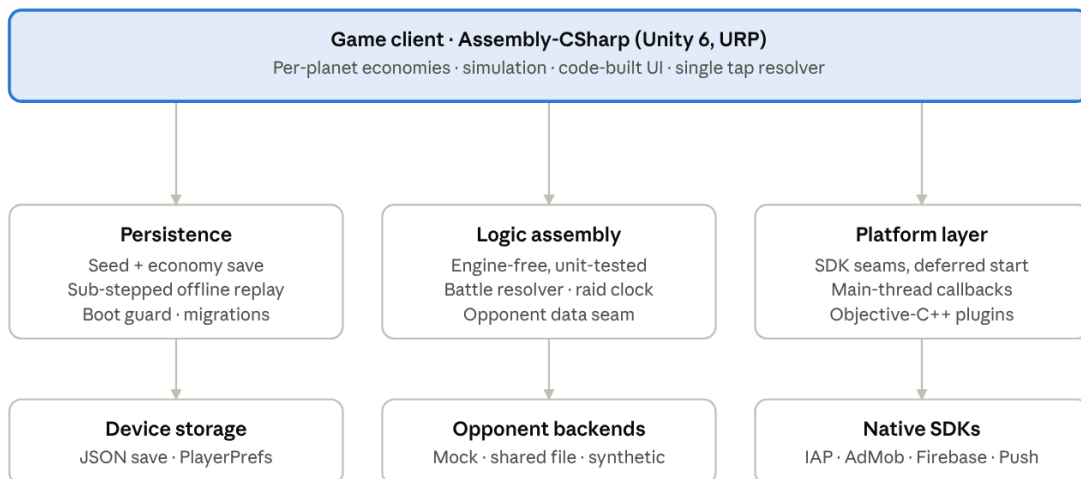
- **Architecture and design:** Designed and planned 46 milestones of progression to make the game entertaining and slowly increasing in complexity to allow the users to get a natural feel of the controls and actions. Each planet in the game had to have its own inventory of materials, workers, buildings, production, all ticking according to passed time even when the application is closed. Some battle aspects of the game had to be isolated with plain C# rather than depending on Scene states to create fast unit tests without having to run the game.
- **Gameplay and simulation:** Built 34 interlocking systems: gathering, smelting and factory crafting chains, with 12 research tree nodes with a randomized dependency graph. 50+ Panels written in code alone.
- **Persistence and offline progress:** Built a seed-plus-economy save format with atomic writes, a boot guard that protects saves from crash loops. Offline catch-up replays the real production code in sub-steps instead of estimating it.

- **Localisation:** Built a 29-language layer with a glossary of game nouns, effectively using Anthropic Claude's LLM API, with Claude Code's context to give a better meaning to the translations and to store in a local file the output that maps each language translated to the original English wording.
- **Monetization and platform:** Integrated Unity IAP, Google Mobile Ads with UMP consent and iOS App Tracking Transparency, local push notifications, Firebase Crashlytics, store review prompts and native Objective-C++ plugins.

Technical Architecture

The game is one Unity client split into a gameplay layer, an engine-free logic assembly, and a thin platform layer that is the only code allowed to touch native SDKs.

Gameplay code reaches native SDKs and backends only through seams



Game client (Assembly-CSharp): Simulation, rendering and UI for the whole galaxy.

- **Implementation:** Unity 6 (6000.5) with the Universal Render Pipeline, C#. Nothing is authored in the scene: a bootstrap builds the camera, planets and every panel at runtime.
- **World:** procedurally generated planets from a saved seed, with custom HLSL shaders. An editor script keeps every runtime so nothing is stripped from device builds.
- **Economy:** Each planet owning its own inventory, populations and buildings. Production runs on a shared time cycle.
- **UI:** code-built Canvas UI with manual pointer hit-testing, a shared blocker registry, and explicit sorting-order bands for tabs, panels and modals.

Persistence (Scripts/Save): Durable world state and offline progression.

- **Format:** JsonUtility save of the master seed plus the economy; the world is regenerated from the seed, then the economy poured back in.

- **Safety:** atomic temp-file writes, and a boot guard that sets a save aside only after three consecutive failed launches.
- **Offline:** a sub-stepped replay of the real producers in chain order, a craft sub-step, with an idle-time bank.

Platform layer:

- **Monetization:** Unity IAP 5 with live localized store prices and a Restore Purchases flow; Google Mobile Ads with UMP consent, App Tracking Transparency, and SDK start deferred until after the first rendered frame.
- **Services:** Unity Mobile Notifications (scheduled raid and research alerts), Firebase Crashlytics, StoreKit and Play review prompts.

Scale & Constraints

- **Two platforms, one codebase:** Every build and configuration change had to land on iOS and Android together, across Xcode, Gradle, two ad and purchase stores, and two review processes.
- **Idle-game time scales:** Production ranges from seconds to 24 hours, and the game must credit hours of absence correctly when the app is closed, backgrounded or killed by the OS without warning.
- **Save durability:** A player's galaxy represents weeks of play. A launch-time crash, an iOS watchdog kill or a format change must never cost it.
- **29 languages and four scripts:** Right-to-left Arabic, Persian, Urdu and Hebrew needed contextual shaping and reordering that Unity's legacy text system does not provide, and CJK and Thai needed font fallbacks.
- **Store and privacy compliance:** Apple App Review, EU consent (UMP), App Tracking Transparency, and Google Play rules on alarms and permissions.
- **Solo development:** One developer owned design, engineering, balance, art integration, localisation and release, so every system needed to be self-checking.

Engineering Challenges

- **Rendering cost versus logical scale:** Drawing every unit kills frame rate, yet a 400-soldier garrison must fight like 400, and 4,000 workers must gather like 4,000.
- **Tap arbitration without an EventSystem:** Tapping on different assets on a globe (planet) created issues where the tap target could select an asset on the other side of the globe. The solution to this was to have limitations on where buildings could be made and each building is surrounded by a sphere like structure that acts as the main target clicks with the globe (planet) itself consuming the tap action if a misclick happens.
- **Main-thread safety with native SDKs:** Firebase, ads, purchases and push all call back on their own threads. One wrong continuation deadlocked the app at launch with no crash to report.

Approach & Implementation

- **Replayed the real code instead of estimating it.** Offline catch-up drives each planet's producers in explicit chain order (gather, smelt, craft, spores) over bounded sub-steps.

- **Separated what is drawn from what is counted.** Each unit pool has a render budget (60 workers, 50 soldiers per planet) and banks the rest as an integer reserve. Rates read the true count, and each visible soldier fires for its share of the garrison, so 4,000 workers gather like 4,000 while 60 are drawn. The number drawn depends on the graphic settings in the settings menu with Low/Medium/High.

Outcome

- **Performance:** A planet at every render cap draws a fixed 260 roamers however many it logically holds.
- **Reliability:** Offline production matches live play to within 2 to 6% for the exotic-material craft, and transport loops match live throughput across all tested wait times.
- **Save safety:** Atomic writes and boot guard with in-game restore mean a crash at launch no longer costs a player their world.
- **Reach:** 29 languages, including four right-to-left scripts, from about 945 keys.
- **Compliance:** Purchases became visible and restorable on a fresh install, EU consent and App Tracking Transparency were wired in, and SDK startup was moved out of the boot path so a third-party crash cannot cost a save.

-